Revvel Personal Ops Fleet

Architecture and Operations Handbook

A proposal-first control plane for a hybrid personal operations agent fleet: scored proposals, explicit policy decisions, reversible-first actions, and an append-only hash-chained audit trail.

Field	Value
Document	Revvel Personal Ops Fleet — Architecture and Operations Handbook
Author	Perplexity Computer
Version	0.1.0 (MVP)
Date	August 03, 2026
Repository	revvel-personal-ops-fleet
Status	MVP — plan and dry-run only; no execution path implemented
Scope	Design + operations. Contains no secrets, credentials, memory contents, system prompts, or proprietary tool internals.
Sources	None. Original design document; no external research or citations.

> **Read this first**
>
> Every action in this system starts life as a **proposal**, receives a **0-100 confidence score**, and receives exactly one **policy disposition**: allow, propose, require_approval, or deny. High-risk, irreversible, and externally visible actions are blocked for human approval by default. Nothing in the MVP can mutate an external service.

00 Contents

01 System overview

The fleet is a **decision layer** for personal operations across mail, calendar, files, code, and existing automations. It converts messy inputs into a queue of scored, policy-gated, reversible proposals, each backed by evidence and a rollback reference. Execution is the last, smallest, and most heavily gated part of the system — and in this MVP it is absent by design.

```
plan  ->  score (0-100)  ->  policy disposition  ->  dry-run  ->  [human approval]  ->  apply
                                                                                        ^ refused in MVP
```

Enforced invariants

Invariant	Enforcement point
No module performs a network call	Connectors are skeletons; `apply()` raises NotImplementedError
Default autonomy is review_everything	Policy config defaults and PolicyConfig model
High-risk / irreversible / externally visible actions need a human	Policy rules R050, R060, R061
Delete and unsubscribe always need a human	Rule R040 + always_require_approval_permissions
Deletion means Trash, never permanent delete	Email cleanup skill + capability deny list
Only allowlisted identities are addressable	Rules R020 / R021
Mutating automation requires a rollback reference	Rule R070
Audit log is append-only and tamper-evident	Append-mode writes + SHA-256 chain verification
Apply is refused	Planner, CLI, and API all refuse and record the refusal

Demonstrated behavior

Running the offline demo over ten synthetic mail threads produces 23 proposals across four action families — label, archive, unsubscribe proposal, and Trash proposal — and gates them:

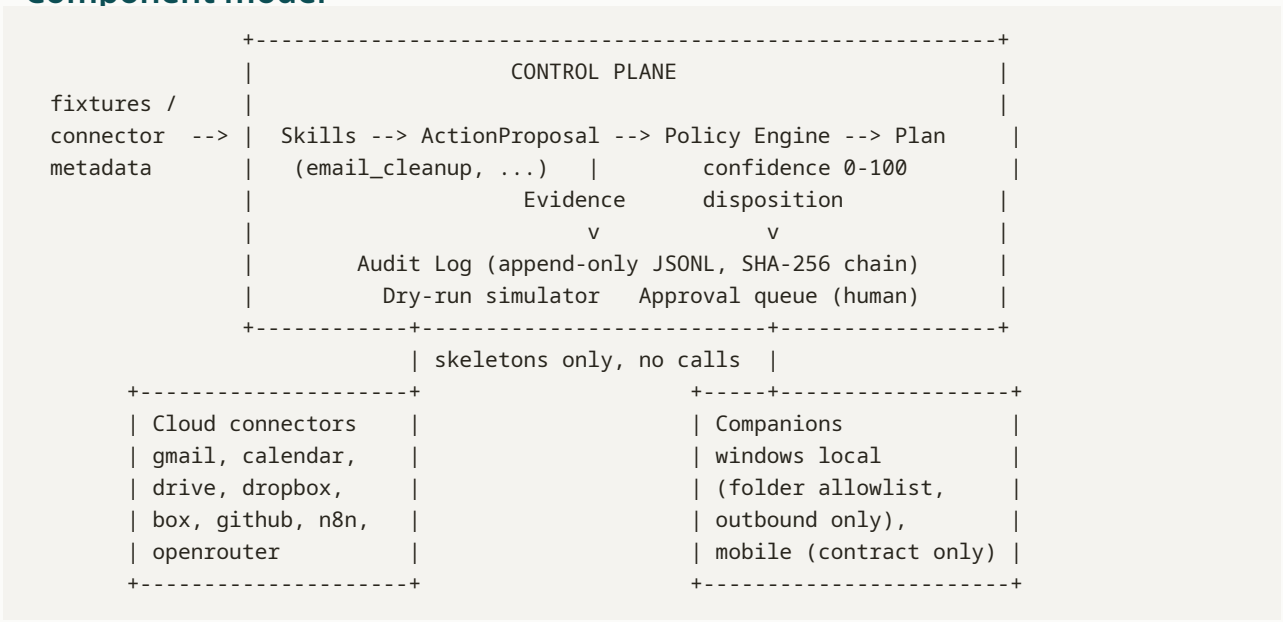
Autonomy mode	allow	propose	require_approval	deny
review_everything	0	0	16	7
safe_automation	0	16	0	7
policy_automation	0	16	0	7

> **Why allow is zero even at confidence 100**

> The seven denials are the delete and unsubscribe proposals: the operator identity is granted read, suggest, and write — not delete or unsubscribe — so they are refused at the identity layer before scoring matters. And although labeling scores 100, the connector's declared gate for label application is **propose**. A connector gate may only tighten a decision, never loosen it.

02 Architecture

Component model



Module responsibilities

Module	Responsibility	Never does
models	Typed domain objects and hashing helpers	Persist or transmit
config	Load policy / identities / storage YAML with restrictive defaults	Read secrets
skills	Turn metadata into proposals with evidence	Execute or fetch
policy	Score 0-100, assign disposition, enforce rule order	Mutate targets
planner	Orchestrate plan -> dry-run -> refused apply; write audit	Bypass policy
audit	Append-only JSONL, hash chain, redaction, verification	Rewrite history
inventory	Labeled sample connector state and capability matrix	Probe live services
connectors	Declare capabilities, scopes, gates, rollback	Open a socket
cli / api	Operator surfaces	Expose an apply path

Data flow for one run

1. **Ingest** — a skill receives already-fetched metadata (a fixture in the MVP). Message bodies are never loaded, so they cannot leak into evidence or logs.
2. **Propose** — proposals default to the unsafe interpretation (high risk, irreversible, externally visible) until the skill proves otherwise.
3. **Evidence** — each row records connector, opaque source reference, signal name, and a SHA-256 digest of the redacted observation. Evidence IDs are referenced from the audit log.
4. **Score** — evidence count, reversibility, external visibility, rollback availability, a bounded skill heuristic, minus risk and permission penalties.
5. **Decide** — ordered rules return exactly one disposition; a connector gate may then only tighten it.
6. **Record** — plan started, fixture loaded, proposal created, policy decided, plan completed, each linked by prev_hash.
7. **Dry-run** — non-denied items are simulated; the report asserts zero network calls.
8. **Approve** — queued items carry approvers and an expiry (default 12 hours).
9. **Apply** — refused in the MVP; the refusal itself is an audit event.

Trust boundaries

Boundary	Direction	Control
Control plane <-> cloud APIs	outbound only	Least-privilege scopes, identity allowlist, capability gates
Control plane <-> Windows companion	companion dials out; no inbound port	Folder allowlist, local approval prompt, enrollment token in OS credential store
Control plane <-> mobile companion	device-initiated push only	OS permissions + device enrollment + biometric local authorization
Control plane <-> n8n	outbound to a registered workflow allowlist	Production triggers are approval-gated; n8n credentials stay in n8n
Control plane <-> model routing	outbound, redacted payloads only	Raw-content completion is hard-denied; spend and token caps
Audit store	write-append, read-verify	Hash chain; retention exceeds every provider trash window

State and storage

- **var/audit/<run_id>.jsonl** — the chain. One file per run, append-only, fsynced per event.
- **var/plans/<plan_id>.json** — serialized plan (proposals plus decisions) for approval review.
- **var/inventory/<inventory_id>.json** — labeled sample inventory snapshots.
- No database. Files are the source of truth; every persisted object has a generated JSON Schema.

03 Safety model

Assets: mail and calendar metadata (never bodies), file metadata across four stores, the audit chain's integrity, and identity bindings. Credentials live outside the repository.

Threats and mitigations

Threat	Mitigation	Residual risk
Agent acts over the wrong signed-in account	Identity allowlist with explicit deny rows; rules R020 / R021	Operator misconfigures the allowlist
Runaway bulk mutation	Dry-run default, per-run caps, approval on all delete and unsubscribe	An approved batch is still large
Irreversible external side effect	Externally visible and irreversible always gated; worst cases denied	A human approves a bad proposal
Audit tampering	SHA-256 chain, append-only writes, verifier plus tamper demo	Whole-file deletion (offsite replication is backlog)
Secret leakage into logs or docs	Redaction, gitignore, secret references only, matrix test for secret-like strings	A human pastes a secret into config
Local companion abused as a remote shell	Outbound-only, empty command allowlist, forbidden roots, local prompt	Operator adds a dangerous command
Mobile over-collection	Nothing readable by default; OS permission + enrollment + biometric	Operator opts into notification forwarding
Third-party model retention	Redacted payloads only; raw-content path denied	Provider-side retention policy
Prompt injection from content	Policy is deterministic code over typed fields; no natural language is executed	Future content-reading skills must re-establish this

Data minimization and redaction

Redaction runs before any payload is written. Keys named body, content, snippet, text, html, message, token, access_token, refresh_token, password, secret, api_key, or authorization are replaced with a digest reference. Evidence stores an opaque source reference plus a digest, never the observation itself, and notes are capped at 280 characters to discourage copying content into logs.

> **Prompt-injection stance**
> Untrusted text may influence which category a skill suggests, but it can never change a disposition. The policy engine reads only typed fields — permission verb, risk tier, reversibility, external visibility, identity, capability — plus a bounded integer heuristic. There is no path from message text to a policy override.

Never present in this repository

Secrets, tokens, OAuth client secrets, cookies, session material, private email addresses beyond the single allowlisted operator account, real message contents, agent memory contents, raw internal system prompts, or closed/proprietary tool internals. Only example configuration templates are tracked; live configuration, environment files, and run artifacts are ignored by version control.

04 Policy engine

Autonomy modes

Mode	Meaning	Behavior
review_everything	Default. Nothing executes without a human.	Every non-denied proposal becomes require_approval (R030).
safe_automation	Auto-execute only an explicit allowlist of reversible, internal-only capabilities.	Capability must be allowlisted and score at or above the allow threshold; otherwise propose.
policy_automation	Threshold-driven, still bounded by every hard rule.	>=90 allow; >=55 propose; below that require_approval.

Modes never widen the hard rules. Raising the mode can only affect proposals that already passed identity, permission, risk, reversibility, external-visibility, and rollback checks.

Confidence score (0-100)

```
score = 20 (base)
      + min(12 * evidence_count, 36)
      + 20 reversible | 8 conditionally reversible
      + 12 if not externally visible
      + 10 if a rollback reference exists
      + round(0.30 * skill heuristic)
      - risk penalty        {low 0, medium 10, high 25, critical 45}
      - permission penalty  {read 0, suggest 0, write 8, unsubscribe 18, delete 30}
      clamped to [0, 100]
```

Every term is recorded in the decision's reason list, so a score is always explainable. Scoring and gating are separate axes: **a high score never unlocks a gated capability**.

Rule table (first match wins)

Rule	Condition	Result
R000	always	compute confidence
R010	capability on the deny list	deny
R020	identity not allowlisted	deny
R021	permission not granted to that identity	deny

Rule	Condition	Result
R030	mode is review_everything	require_approval
R040	permission is delete or unsubscribe	require_approval
R041	capability on the always-approve list	require_approval
R050	action is externally visible	require_approval
R060	action is irreversible	require_approval
R061	risk tier is high or critical	require_approval
R070	mutating action with no rollback reference or procedure	require_approval
R080-R082	safe_automation: allowlist and threshold checks	allow or propose
R090-R092	policy_automation: threshold bands	allow, propose, or require_approval
R100	connector-declared gate is stricter than the decision	tighten to the connector gate

Reversible-first doctrine

1. Prefer the reversible variant: label over archive, archive over trash, trash over permanent delete, staging move over local delete, draft over send.
2. A mutating action without a rollback reference cannot be automated (R070).
3. The rollback reference must be resolvable before execution — a label snapshot, an inbox restore reference, an untrash reference, a prior parent folder, a branch reference to delete.
4. Audit retention (400 days) must exceed every provider trash window, so the record outlives the ability to undo.

Approvals

- Approval-bearing decisions carry required approvers and an expiry (default 720 minutes).
- Expired decisions are re-planned and re-scored, never revived: inputs may have changed.
- Approvals are recorded as audit events; they can be superseded but never erased.
- Batch approval is acceptable only when every item shares one capability and one rollback path.

Configuring policy safely

- Keep the safe-automation allowlist short, reversible, and internal-only.
- Never move a capability off the deny list without an architecture decision record.
- Lowering the allow threshold below 90 requires an ADR and a changelog entry.
- Re-run the test suite after every change: it asserts that delete, unsubscribe, externally visible, high-risk, and rollback-less actions are never auto-allowed in any mode.

05 Identity and account selection

The most likely serious incident is an agent acting correctly on the **wrong identity**. Multiple accounts are typically signed in on the same machine and browser, so identity binding is the primary brake.

Policy

- There is no ambient credential path: every proposal carries an explicit identity.
- Identities are allowlisted per provider with **separated permission verbs**: read, suggest, write, delete, unsubscribe. Granting read never implies write.
- A non-allowlisted identity is denied at R020; a permission not granted to that identity is denied at R021 — both before scoring matters.
- An entry present with allowlisted set to false is an **explicit denial**. This is how other signed-in accounts are neutralized.
- Credentials themselves live in the OS keychain or a secret manager and are referenced by name only.

Example allowlist shape (illustrative, non-secret)

Identity	Provider	Allowlisted	Permissions
angelreporters@gmail.com	google (mail + calendar)	yes	read, suggest, write
angelreporters@gmail.com	google_drive	yes	read
other-google-account@example.com	google	**no — explicitly forbidden**	—
revvel-ops-bot	github	yes	read, suggest
dropbox:primary	dropbox	yes	read
box:primary	box	no — disabled by operator choice	—

> **Two independent brakes on the dangerous verbs**
>
> The operator identity is granted write for reversible, internal-only mutations such as labeling and archiving, and is deliberately **not** granted delete or unsubscribe. Those proposals are therefore refused twice over: once at the identity layer (R021) and once at the policy layer (R040). Widening a grant is a reviewable configuration diff, never a runtime decision.

Service accounts

Service or bot identities (for example a GitHub automation identity) are allowlisted separately and constrained further at the connector layer by repository, folder, or workflow allowlists. Never reuse a human identity as a service identity, and never grant a service identity delete or unsubscribe.

06 Connector contracts

Every connector is a skeleton adapter that declares capabilities and refuses to execute. Each capability names a permission verb, minimum provider scopes, risk tier, reversibility, external visibility, an approval gate, a rollback statement, and an honest availability value: available, partial, planned, or unavailable. Every right-hand side — identities, scopes, allowlists, gates, spend caps — is user-configurable.

Permission verbs are never bundled

Verb	Meaning
read	Fetch metadata or state. No mutation.
suggest	Produce proposals locally. Touches nothing external.
write	Mutate state reversibly or with a defined rollback.
delete	Remove or trash. Always approval-gated; permanent deletion is denied.
unsubscribe	Contact a third party. Externally visible, irreversible, always approval-gated.

Gmail and Calendar

Capability	Verb	Scope	Gate	Rollback
messages.search	read	gmail.readonly	allow	n/a
messages.categorize	suggest	—	allow	discard proposal
labels.apply	write	gmail.modify	propose	remove the fleet-namespaced label
threads.archive	write	gmail.modify	propose	re-add INBOX
threads.trash	delete	gmail.modify	require_approval	untrash within retention
subscriptions.unsubscribe	unsubscribe	gmail.readonly	require_approval	none — manual resubscribe
messages.send	write	gmail.send	require_approval	none
calendar.events.list	read	calendar.readonly	allow	n/a
calendar.create_private_hold	write	calendar.events	propose	delete the created event
calendar.invite_guests	write	calendar.events	require_approval	cancellation only

> **Do not assume all Gmail operations are available**
>
> The reachable operation set depends both on granted scopes and on what the host platform's Gmail connector actually exposes at runtime. Several capabilities are marked planned or partial for exactly that reason and may require an independently authorized OAuth client. Permanent deletion, filter mutation, and forwarding rules are out of scope entirely.

File stores

Connector	Sample status	Read scope stance	Gated writes
Google Drive	connected — ready to revalidate	Metadata-readonly or drive.file only; full-drive scopes are never requested	Move proposes; sharing and trash require approval
Dropbox	connected	files.metadata.read under a path allowlist	Move proposes; shared links and delete require approval
Box	**disabled — reconnect required, by user choice**	All capabilities resolve to unavailable	None reachable; do not retrigger authorization
Local (Windows)	not configured	Metadata and hashes only, inside an explicit folder allowlist	Staging move proposes; delete is denied outright

> **Current connector state — labeled sample, revalidate at runtime**
>
> Gmail, Calendar, GitHub, and Dropbox are recorded as connected with read and suggest permissions. **Google Drive authorization has succeeded** and is recorded as connected — treat it as ready-to-revalidate rather than proven, because authorization is not access: the folder allowlist is still empty. **Box authorization was dismissed by the operator**, so Box is recorded as disabled with reconnect required; the fleet must never retrigger a Box authorization prompt, and reconnect is operator-initiated only. The local and mobile companions, n8n, and model routing are not configured. **Every status value here is a non-secret sample and must be revalidated at runtime.**

GitHub, n8n, and model routing

Capability	Verb	Gate	Notes
github.repos.read	read	allow	Repository allowlist; fine-grained token or app only
github.issues.propose	suggest	allow	Drafts locally; nothing is posted
github.branches.create	write	propose	Rollback deletes the branch reference
github.pulls.create	write	require_approval	Externally visible; notifications cannot be recalled
github.repos.delete	delete	**deny**	Hard-denied. No repository is ever created or modified here

Capability	Verb	Gate	Notes
n8n.workflows.list / executions.read	read	allow	Inventory and reliability metrics
n8n.trigger_dry_run	suggest	propose	Only workflows declared side-effect free
n8n.trigger_production	write	require_approval	Compensating flow required before enabling
n8n.workflows.modify	write	require_approval	Rollback restores an exported workflow snapshot
model routing — redacted completion	suggest	propose	Minimized, redacted input; token and spend caps
model routing — raw content	write	**deny**	Sending unredacted personal content is denied

07 Companions

Windows local companion

A user-installed per-user tray application. It is **outbound-only**: it dials the control plane over TLS, never listens on a port, and is not remotely addressable. It operates strictly inside an explicit, absolute folder allowlist — no wildcards above an allowlisted root, no system directories. An empty allowlist means it does nothing.

Design element	Rule
Folder allowlist	Explicit absolute paths only. The companion refuses to start if the allowlist includes the Windows directory, Program Files, or a user profile root.
Enrollment	One-time operator-generated pairing code; the resulting token is stored in the OS credential store, never in a file in the repository.
Reads	Metadata only — path, size, modified time, content hash. File contents never leave the machine.
Writes	Reversible staging moves only, and each requires a **local** approval prompt on the machine in addition to control-plane approval.
Deletion	Not implemented and hard-denied. Staging plus retention replaces deletion.
Command execution	Signed, user-defined allowlist that is empty by default. Arbitrary shell execution is never enabled; each entry needs a documented compensating action.

Mobile companion — contract, not an implementation

> **A phone cannot be read by default**

> Nothing in this system can observe a mobile device. Mobile participation requires all three of: (1) an **installed companion app** that the user chose to install, (2) **user-approved OS permissions** granted per data class in the operating system settings and revocable at any time, and (3) a **separate local authorization** — device enrollment with a pairing code plus device biometric confirmation, distinct from any cloud OAuth grant. Absent any one of these, every capability resolves to unavailable.

Capability	Verb	Gate	Availability
capture.push_note	suggest	allow	planned — device-initiated only
approvals.respond	write	require_approval	planned — biometric confirmation per approval
notifications.summarize_optin	read	require_approval	unavailable — Android-only permission class, off by default
files.upload_selected	read	require_approval	unavailable — OS document-picker scope only
device.read_arbitrary	read	**deny**	unavailable — explicitly not offered

The channel is device-initiated push only: there is no server-to-device pull path for user data. iOS restricts background notification and file access far more than Android, so assume less capability, never more. Revoking an OS permission or unenrolling the device disables the capability immediately.

08 Operational workflows

Daily loop

When	Action	Command
Session start	Confirm posture and thresholds	revvel-ops doctor
Before planning	Revalidate connector state and identity grants	revvel-ops inventory --demo, then a cheap live read
Plan	Build scored, gated proposals	revvel-ops plan --mode review_everything
Simulate	Confirm zero external calls	revvel-ops dry-run
Review	Work the approval queue	read the plan JSON in var/plans
Close out	Prove chain integrity, record the head hash	revvel-ops verify-audit

Email cleanup pass

1. Check posture and identity grants; confirm the intended verbs are the only ones granted.

2. Plan in review_everything. Expect four families: label on every thread, archive for stale bulk mail, unsubscribe for high-volume low-engagement senders, and Trash for stale promotional or notification threads only.
3. Protected categories — security, personal, receipt — are never archived or trashed.
4. Dry-run and verify zero network calls plus an intact chain.
5. Review the plan file item by item: identity, target, evidence, rollback reference, and the rule identifiers behind the disposition.
6. Approve labels first. Never batch-approve unsubscribe or Trash items.
7. Apply is unavailable in the MVP and records a refusal event. When live adapters exist, apply labels, verify, then archive, then re-review deletions.

Approval triage order

- **deny** — read the rule identifier; if a denial is wrong, fix configuration or the skill. Never bypass.
- **require_approval** — decide individually. Reject anything whose rollback needs more than one sentence.
- **propose** — batch only within one capability and one rollback path.
- **allow** — spot-check that the capability really is reversible and internal-only.

Incident response

1. Contain: set autonomy to review_everything and stop planning. Never edit or delete anything under the artifact directory.
2. Preserve: copy the audit directory to a read-only location; work only on copies.
3. Verify: recompute the chains. Intact means the record is trustworthy; broken means treat the host as untrusted and rotate referenced credentials.
4. Reconstruct: the policy decision event carries confidence, disposition, rule identifiers, and reasons; the paired proposal event carries identity, capability, target, and evidence digests.
5. Classify: policy followed but outcome undesired (tighten gates), policy not followed (code defect — failing test first), wrong identity (allowlist error), or action outside the fleet (inventory the other automation).
6. Recover: execute the documented rollback and append it to the chain.
7. Postmortem: timeline, evidence identifiers, rule identifiers, root cause, the configuration or code change, and the new test. Reference identifiers, never content.

09 Audit and evidence

Chain construction

```
payload_hash = SHA256(canonical(payload))
event_hash   = SHA256(event_id, sequence, recorded_at, run_id, phase, event_type,
                      actor, identity, subject_ref, payload_hash, prev_hash)
first event   prev_hash = 64 zeros
```

Canonical encoding sorts keys and normalizes timestamps to a single UTC form, so re-reading a line can never change its hash. Writers open in append mode and flush to disk per event; reopening a run resumes the chain from the last line. Signature and signer fields exist on every event and are always empty in the MVP: hash chaining is claimed, non-repudiation is not.

What verification checks

- Sequence numbers are monotonic from zero with no gaps.
- Each event's previous-hash equals the prior event's hash; the first links to the genesis value.
- Each payload hash matches a recomputation of its payload.
- Each event hash matches a recomputation over the sealed field set.

Failure interpretation

Error	Likely cause	Action
payload hash mismatch	A payload was edited in place	Incident: preserve and investigate
event hash mismatch	Header field edited, or code version skew	Check version; otherwise incident
previous-hash break	A line was inserted, removed, or reordered	Incident: reconstruct from the last good sequence
sequence mismatch	Concurrent writers or a manual edit	Ensure one writer per run

Evidence discipline

- Every proposal carries evidence rows: connector, opaque source reference, signal name, SHA-256 digest, and a note capped at 280 characters.
- Evidence identifiers and digests are the audit currency — reference them in reviews, tickets, and postmortems instead of pasting content.
- If a decision cannot be justified from its evidence rows, that is a defect in the skill, not a reason to approve.

- Retention is 400 days and must exceed every provider's trash window. Archive whole files by run; never truncate.

> **Tamper evidence is demonstrable, not asserted**
> A bundled script copies a real chain to a temporary location, edits a single payload, and shows verification flipping from intact to broken with the offending line reported. The original log is never modified. Whole-file deletion remains undetectable until offsite append-only replication ships.

10 Runtime capability boundary

An agent fleet has two very different kinds of ability, and conflating them produces both security mistakes and false expectations. This handbook documents only the first kind.

Dimension	Documented runtime skills (in scope)	Platform-internal capabilities (out of scope)
Definition	Declared capabilities of this control plane and its connectors	Whatever a hosting agent platform can do internally
Where defined	Connector manifests, skill modules, the capability manifest	Vendor-internal; deliberately not reproduced
Contract	Stable capability key, permission verb, scopes, gate, rollback, availability	No contract exposed to this system
Auditable here	Yes — every use is an audit event	No
Policy-gated here	Yes	Not by this policy engine

> **The boundary rule**
> If a capability cannot be named with a permission verb, a least-privilege scope, an approval gate, and a rollback, it is **not** a runtime capability of this fleet, and the fleet must not depend on it. A platform feature that might exist is treated as absent: unknown capabilities are rejected at plan time and recorded as such.

Deliberately excluded

- Raw internal system prompts or instruction text of any agent platform.
- Actual agent memory contents or memory-store internals.
- Closed or proprietary tool internals, private API shapes, undocumented endpoints.
- Secrets, tokens, cookies, session material, and private addresses beyond the single allowlisted operator account.
- Real user data of any kind — every fixture in the repository is synthetic.

Interface contract with a host platform

1. The platform hands over already-fetched metadata, or invokes the command-line/API surface.
2. The fleet returns proposals, decisions, and audit references — never raw user content.
3. Execution, if ever enabled, happens through a declared connector capability with an approval token; the platform's own internal abilities remain outside this policy engine's claims.

Memory boundary

The fleet keeps three durable artifacts only: audit chains, plans, and inventories. It stores no conversational memory, no user profile, and no embeddings. Any future memory feature must arrive as a declared capability with its own retention, redaction, and deletion runbook — never as an implicit side effect.

11 Implementation steps

Phase 0 — shipped in this MVP

- Domain models and generated JSON Schemas for every persisted object.
- Policy engine: 0-100 scoring, four dispositions, ordered rules, three autonomy modes.
- Append-only hash-chained audit log with redaction, verification, and a tamper demonstration.
- Email cleanup skill: deterministic categorization plus label, archive, unsubscribe, and Trash proposals with evidence.
- Ten connector skeletons with scopes, gates, rollback statements, and honest availability.
- Identity allowlist with separated permission verbs, enforced ahead of scoring.
- Command-line interface, optional read/plan HTTP surface with no apply endpoint, test suite, and this handbook.

Sequenced phases and exit criteria

Phase	Work	Exit criterion
1 — read-only reality	Runtime revalidation probes; live metadata ingestion; Drive read inside a non-empty folder allowlist; keychain-backed secret references; approval queue with expiry	Each scope proven by a cheap read recorded as an audit event; no secret material in files or logs
2 — first reversible automation	Live label adapter with snapshot rollback; promote one capability to safe automation; per-run batch caps; archive adapter	Rollback restores exactly; two clean weeks of review-mode history before promotion; ADR filed

Phase	Work	Exit criterion
3 — companions	Windows enrollment and outbound channel; allowlist enforcement with forbidden roots; metadata indexing and staging moves with local prompts; mobile enrollment and approvals	No inbound listener; companion refuses system or profile roots; biometric per mobile approval; no device data pulled
4 — integration breadth	n8n workflow registry and dry-run triggers; redacted model routing with spend caps; GitHub read and issue drafting; Dropbox dedupe proposals	Only allowlisted side-effect-free workflows triggerable; raw-content model path still denied; no repository created or modified
5 — assurance hardening	External signer over event hashes; offsite append-only replication; API authentication; multi-approver policy; property-based policy tests	Signatures verified; whole-file deletion detectable; no unauthenticated route; no generated proposal reaches allow while high-risk, external, or irreversible

Escalation ladder, not a switch

Prove a capability in review_everything for at least two weeks of real plans. Review the false-positive rate per capability from the audit log. Promote exactly one reversible, internal-only capability into safe automation with an ADR and a changelog entry. Watch for a week. Only then consider threshold-driven automation — and never for delete, unsubscribe, send, share, or pull-request creation.

12 Known limitations

Stated plainly, because an honest limitation list is a safety feature. Each item is tracked in the repository backlog.

#	Limitation
1	Apply is not implemented anywhere. The fleet cannot execute a single external action.
2	All connectors are skeletons: no client libraries, no auth flows, no retry or rate-limit logic.
3	Signing is not implemented. Integrity rests on SHA-256 chaining plus filesystem controls; non-repudiation is not claimed.
4	The approval queue is a JSON file reviewed by hand. There is no user interface and no notification path.
5	Companion applications for Windows and mobile are designs and contracts only — no binaries exist.
6	The HTTP surface has no authentication and must remain bound to loopback.
7	Categorization is rule-based on purpose: reproducible and auditable rather than clever, so nuanced misclassification is expected.
8	Every connector status recorded in the repository is a labeled sample requiring runtime revalidation.
9	Retention and rotation of local run artifacts are manual; offsite append-only replication has not shipped.
10	Single-operator assumption throughout: no multi-approver policy, no delegation, no role separation.

#	Limitation
11	Google Drive is authorized but its folder allowlist is empty, so no Drive proposal can be planned yet.
12	Box is disabled by operator choice; its capabilities are unavailable and authorization must not be retriggered.

> **Closing principle**
>
> The value of this system is not how much it automates — it is that every action it proposes can be explained, gated, reversed, and proven after the fact. Automation is earned one reversible capability at a time.

Revvel Personal Ops Fleet — Architecture and Operations Handbook · Version 0.1.0 (MVP) · August 03, 2026 · Author: Perplexity Computer · Original design document, no external sources.